week10 예습과제

다중 선택	week10
다중 선택 1	

5.0 개요

자연어 처리 (NLP)

- **자연어(Natural Language):** 인공적으로 만든 프로그래밍 언어와 달리, 사람들이 언어 활동을 위해 자연히 만들어진 언어
- **자연어 처리(Natural Language Processing):** 컴퓨터가 인간의 언어를 **이해·해석·생성**하기 위한 기술
  - 인공지능의 하위 분야, 언어의 구조·의미·맥락을 분석하는 알고리즘과 모델 개발이 목표
- **자연어 처리에서 해결해야 할 문제**

문제	설명
모호성 (Ambiguity)	단어·구가 쓰인 맥락에 따라 여러 의미 → 다양한 의미를 구분할 수 있어야 함
가변성 (Variability)	사투리, 강세, 신조어, 작문 스타일 등으로 매우 가변적
구조 (Structure)	구문(Syntactic)을 파악해 의미(Semantic)를 해석 → 문장 구조와 문법 요소 이해 필요

말뭉치와 토큰

- **말뭉치(Corpus):** 자연어 모델을 훈련·평가하는 데 사용되는 **대규모 자연어 텍스트 데이터** (뉴스 기사, 사용자 리뷰, 칼럼 등)
- **토큰(Token):** 말뭉치보다 작은 단위 (개별 단어, 문장 부호, 기타 의미 단위)
- **토큰화(Tokenization):** 컴퓨터가 자연어를 이해할 수 있도록 말뭉치를 토큰으로 나누는 과정
- **토크나이저(Tokenizer):** 텍스트 문자열을 토큰으로 나누는 알고리즘 또는 소프트웨어
- 토큰을 나누는 기준에 따라 시스템 성능·처리 결과가 크게 달라짐

토크나이저 구축 방법

- **공백 분할:** 공백 단위로 분리
- **정규표현식 적용:** 특정 패턴을 식별해 분할
- **어휘 사전(Vocabulary) 적용:** 사전에 정의된 단어 집합을 토큰으로 사용
- **머신러닝 활용:** 토큰화 방법 자체를 데이터로 학습

> 💡 **어휘 사전 방식의 문제점**

- **OOV(Out of Vocab):** 어휘 사전에 없는 단어/토큰 → OOV 문제를 고려해 토큰화해야 함
- 어휘 사전이 크면 학습 비용 ↑, **차원의 저주(Curse of Dimensionality)** 위험
  - 모든 토큰을 벡터화하면 어휘 수만큼 차원이 필요, 대부분 0인 **희소(sparse) 데이터**
  - 출현 빈도만 고려 → **토큰의 순서 관계**를 잘 표현하지 못함

5.1 단어 및 글자 토큰화

1) 단어 토큰화 (Word Tokenization)

- 텍스트를 **의미 있는 단위인 단어**로 분리 (띄어쓰기, 문장 부호, 대소문자 등 구분자 활용)
- 품사 태깅, 개체명 인식, 기계 번역 등에서 널리 쓰이는 **가장 일반적인 방법**
- 구현: 문자열의 `split()` 메서드 (구분자 미입력 시 공백 기준)
- **한계**
  - '최고!'와 '최고!!', 'cg.'와 'cg'가 서로 **다른 토큰**으로 취급됨
  - 'cg'가 사전에 있어도 'cg.', 'cg는', 'cg도' 등은 **OOV**
  - 한국어 접사, 문장 부호, 오타, 띄어쓰기 오류에 **취약**

2) 글자 토큰화 (Character Tokenization)

- 글자 단위로 문장을 나눔 (구현: `list(review)`, 공백도 토큰이 됨)
- **장점**
  - **작은 단어 사전** 구축 가능 → 컴퓨터 자원 절약, 각 토큰을 더 자주 학습
  - OOV를 획기적으로 감소, 접사·문장 부호('도', '는', '.')의 의미도 학습 가능
  - 언어 모델링 같은 **시퀀스 예측 작업**(다음 문자 예측)에 유용
- **단점**
  - 개별 토큰이 **의미가 없어** 모델이 조합해서 의미를 도출해야 함
  - 다의어·동음이의어가 많은 도메인(문장 생성, 개체명 인식)에서 구별이 어려움
  - **시퀀스 길이 증가 → 연산량 증가**

자소 단위 토큰화

- 한글은 한 글자가 여러 자음·모음의 조합 → **자소(字素) 단위**로 분해
- **자모(jamo) 라이브러리** 사용 (`pip install jamo`)

함수	역할
`jamo.h2j()`	완성형 한글 → **조합형** 한글 자모로 변환 (유니코드 U+1100~U+11FE)
`jamo.j2hcj()`	조합형 한글 → **자소 단위**로 분리 (초성·중성·종성)

- **조합형:** 글자를 자모 단위로 나눠 인코딩 후 조합 / **완성형:** 조합된 글자 자체에 값을 부여
- 사용: `j2hcj(h2j(review))`

> 💡 **예시 문장의 입력 시퀀스 길이 비교**
>
> - 단어 토큰화: **13**
> - 글자 토큰화: **53**
> - 자소 단위 토큰화: **101**

5.2 형태소 토큰화

- **형태소(Morpheme):** 실제로 의미를 가지는 **최소 단위**
- **형태소 토큰화:** 텍스트를 형태소 단위로 나눔 (문법·구조 고려)
- **교착어(Agglutinative Language)** 인 한국어에서 중요: 어근에 접사·조사가 결합해 하나의 낱말을 이룸
  - 예) '그는 나에게 인사를 했다' → 그(+는), 나(+에게), 인사(+를), 했다

구분	자립 형태소 (Free)	의존 형태소 (Bound)
특징	스스로 의미를 가짐	다른 형태소와 조합되어 사용
해당	명사, 동사, 형용사 등	조사, 어미, 접두사, 접미사
예시	그, 나, 인사	는, 에게, 를, 했, 다

형태소 어휘 사전

- 각 단어의 **형태소 정보**를 포함하는 어휘 사전 (품사, 뜻 등 함께 제공)
- 장점: 띄어쓰기 기준 사전이라면 '그는', '나는', '그녀는'을 서로 다른 단위로 인식하지만, 형태소 단위 사전이면 **'그녀'만 새로 학습**하면 '그녀는', '그녀에게'도 쉽게 학습
- **품사 태깅(POS Tagging):** 형태소 분석 후 각 형태소에 **품사(Part Of Speech)** 를 태깅 → 문맥을 고려한 정확한 분석
  - 예) 그(명사) + 는(조사) + 나(명사) + 에게(조사) + 인사(명사) + 를(조사) + 했다(동사)

1) KoNLPy

- **한국어 자연어 처리** 라이브러리 (명사 추출, 형태소 분석, 품사 태깅 등)
- **자바(JDK) 기반**의 형태소 분석기 → **JDK 설치 필요**
- 지원 분석기: **Okt, 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab)**
  - 메캅은 윈도우 미지원 (맥, 우분투만 가능)
- 실습: **Okt**(SNS 텍스트 기반) / **꼬꼬마**(국립국어원 **세종 말뭉치** 기반)

기능	Okt	꼬꼬마
명사 추출	`nouns`	`nouns`
구 추출	`phrases`	× (지원 안 함)
문장 추출	×	`sentences`
형태소 추출	`morphs`	`morphs`
품사 태깅	`pos`	`pos`
품사 개수	**19개**	**56개** (더 세분화)

- 품사 태깅 결과: `(형태소, 품사)` 튜플의 리스트
- 같은 메서드라도 분석기마다 결과가 다름 (예: Okt는 '낼', '수'를 명사로 추출, 꼬꼬마는 '할'을 '하', 'ㄹ'로 더 분리)
- 품사 수 ↑ → 더 자세한 분석 가능하지만 **소요 시간 ↑**, 과도한 분리로 **모델 성능 저하** 가능
- ⇒ **시스템 목적·환경에 맞는 분석기 선택**

2) NLTK

- **Natural Language Toolkit:** 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등 제공
- 주로 **영어**용, 네덜란드어·프랑스어·독일어 등 데이터·모델도 제공
- 필요 모델 다운로드: **Punkt**(통계 기반 토큰화), **Averaged Perceptron Tagger**(퍼셉트론 기반 품사 태깅) → 둘 다 **트리뱅크(Treebank)** 영어 말뭉치로 학습
- `word_tokenize`: 공백 기준 단어 분리 + 구두점 처리 / `sent_tokenize`: 마침표·느낌표·물음표 기준 **문장 분리**
- `pos_tag`: 토큰화된 단어 리스트에 품사 태깅 (**35개** 품사)
- 영어는 한국어보다 토큰화하기 쉬운 구조

3) spaCy

- **Cython** 기반 오픈소스, **머신러닝 기반**으로 **빠른 속도·높은 정확도**가 목표

구분	NLTK	spaCy
목적	학습용 (다양한 알고리즘·예제 제공)	효율적 처리 속도 + 높은 정확도
모델	상대적으로 작음	더 크고 복잡, 더 많은 리소스 필요

- GPU 가속, 24개 이상 언어의 사전 학습 모델 제공 (실습: 영어 `en_core_web_sm`)
- **객체 지향적:** `nlp(sentence)` → **doc 객체** → 여러 **token 객체**
  - `token.pos_`: 기본 품사 / `token.tag_`: 세분화 품사 / `token.text`: 원본 텍스트
  - 그 외 `text_with_ws`, `vector`, `vector_norm` 등
- 형태소 분석·품사 태깅은 키워드 추출, 긍/부정 판단 등 언어 모델 성능에 중요하지만, 띄어쓰기·맞춤법 오류, 신조어, 외래어, 축약어에는 완벽히 대응하기 어려움

5.3 하위 단어 토큰화 (Subword Tokenization)

필요한 이유

- 형태소 분석기의 한계: **신조어, 외래어, 오탈자, 띄어쓰기 오류** 처리 취약
  - 예) '시보리' → '시', '보리' / '어벙하지안구' → '어', '벙하지안구'
  - 예) '돈쭐날만' → '돈쭐', '날', '만'이 적절한데 기존 분석기는 '돈쭐날'로 분해 → 어미 조합마다 별도 토큰 → **어휘 사전 크기 증가**
- **하위 단어 토큰화:** 하나의 단어를 **빈번하게 사용되는 하위 단어(Subword)의 조합**으로 나눔
  - 예) 'Reinforcement' → 'Rein', 'force', 'ment'
  - 효과: 단어 길이 ↓ → 처리 속도 ↑, **OOV·신조어·은어·고유어 문제 완화**
- 방법: **바이트 페어 인코딩(BPE), 워드피스(WordPiece), 유니그램(Unigram)**

1) 바이트 페어 인코딩 (Byte Pair Encoding, BPE)

- **다이그램 코딩(Digram Coding)** 이라고도 함. 원래 **데이터 압축** 알고리즘 → NLP의 하위 단어 토큰화에 사용
- 가장 **빈번하게 등장하는 글자 쌍**을 찾아 **새로운 글자로 치환(병합)** 하는 과정을 반복
  - 종료 조건: 더 이상 반복되는 쌍이 없거나 **정해진 어휘 사전 크기**에 도달
  - 자주 등장하는 단어 → 하나의 토큰, 덜 등장하는 단어 → 여러 토큰의 조합

> 💡 **예시 1: 'abracadabra'**
>
> - 원문 abracadabra
> - Step 1: 'ab' → A → `AracadAra`
> - Step 2: 'ra' → B → `ABcadAB`
> - Step 3: 'AB' → C → `CcadC`
> - 토크나이저로 쓸 때는 치환 대신 **어휘 사전에 추가**

말뭉치 적용 예시

- 빈도 사전: (low, 5), (lower, 2), (newest, 6), (widest, 3)
- 초기 어휘 사전: 글자 단위 `[d, e, l, o, n, s, t, w, i, ...]`
- 반복 과정
  1. 'e'+'s' = **9번**(newest 6 + widest 3)으로 최다 → **'es'** 병합
  2. 'es'+'t' → **'est'** 병합
  3. ... (총 10번 반복) → 'lo', 'low', 'ne', 'new', 'newest', 'wi', 'wid', 'widest' 추가
- 결과: 말뭉치에 없던 'newer', 'wider', 'lowest'도 OOV가 아니라 **'new er', 'wid er', 'low est'** 처럼 토큰화 가능

센텐스피스 (SentencePiece) + 코포라 (Korpora)

- **센텐스피스:** 구글 개발 오픈소스 **하위 단어 토크나이저**, BPE와 유사한 알고리즘, 워드피스·유니코드 기반 알고리즘 지원, 하이퍼파라미터 설정 가능
- **코포라:** 국립국어원·AI Hub 등의 **말뭉치를 쉽게 사용**하게 해주는 라이브러리
- 설치: `pip install sentencepiece Korpora`
- 실습 데이터: **청와대 청원 말뭉치** (2017.8 ~ 2019.3, **433,631개**), `Korpora.load("korean_petitions")`

SentencePieceTrainer 매개변수	의미
`input`	말뭉치 텍스트 파일 경로
`model_prefix`	모델 파일 이름
`vocab_size`	어휘 사전 크기
`character_coverage`	말뭉치 글자 중 토크나이저가 다룰 수 있는 글자의 비율
`model_type`	토크나이저 알고리즘 (unigram / bpe / char / word)
`max_sentence_length`	최대 문장 길이
`unk_id`	OOV를 의미하는 unk 토큰 id (기본 0)
`bos_id`	문장 시작 bos 토큰 id (기본 1)
`eos_id`	문장 끝 eos 토큰 id (기본 2)

- 학습 결과: `.model`(학습된 토크나이저) + `.vocab`(어휘 사전) 파일 생성
- 공백도 특수문자 **'▁'(U+2581)** 로 취급 (예: 'Hello World' → 'Hello▁World' → 'Hello', '▁Wor', 'ld')
- 사용: `SentencePieceProcessor`
  - `encode_as_pieces`: 문장 → 하위 단어 토큰
  - `encode_as_ids`: 문장 → **정수 인코딩** (어휘 사전 ID 값)
  - `decode_ids` / `decode_pieces`: 정수/토큰 → 문자열 복원
  - `get_piece_size`, `id_to_piece`: 어휘 사전 크기 확인 / ID → 하위 단어
- 특수 토큰: `<unk>`(OOV 매핑), `<s>`(문장 시작), `</s>`(문장 종료)

2) 워드피스 (WordPiece)

- BPE와 유사하게 학습하지만 **빈도 기반이 아닌 확률 기반**으로 글자 쌍 병합
- 새로운 하위 단어를 만들 때 이전 하위 단어와 함께 나타날 **확률**이 가장 높은 쌍을 선택
- **글자 쌍 병합 점수**

  $$\text{score} = \frac{f(x,y)}{f(x)\times f(y)}$$

- f: 빈도 함수, x, y: 병합하려는 하위 단어, f(x, y): x와 y가 조합된 쌍의 빈도
- 예) 'e'+'s'는 가장 빈번(9번)하지만 개별 빈도가 높아 점수가 낮음(≈0.06) → 등장은 적어도 개별 빈도가 낮은 쌍(≈0.33)을 **먼저 병합**
- BPE처럼 더 이상 쌍이 없거나 어휘 사전 크기에 도달할 때까지 반복

토크나이저스 (Tokenizers)

- **허깅 페이스** 라이브러리 (`pip install tokenizers`), 워드피스 API로 쉽고 빠르게 구현·학습
- 제공 기능
  - **정규화(Normalization):** 일관된 형식으로 표준화 (공백 제거, 대소문자 변환, 유니코드 정규화, 구두점·특수문자 처리 등)
  - **사전 토큰화(Pre-tokenization):** 토큰화 전에 공백·구두점 기준으로 작은 단위로 분리
- 학습 흐름: `Tokenizer(WordPiece())` → `normalizer = Sequence([NFD(), Lowercase()])` → `pre_tokenizer = Whitespace()` → `train()` → `save()` (JSON 저장, 어휘 사전 + 메타데이터)
- 사용: `Tokenizer.from_file()` + `WordPieceDecoder()`
  - `encode` / `encode_batch` → `.tokens`, `.ids` / `decode`로 복원
  - 워드피스는 하위 단어 앞에 `##` 표기

정규화 클래스	설명
`NFD()`, `NFKD()`, `NFC()`, `NFKC()`	유니코드 정규화
`Lowercase()`	대문자 → 소문자
`Strip()`	양 끝 공백 제거
`StripAccents()`	엑센트 기호 제거
`Replace()`	문자/정규식 기반 변환
`BertNormalizer()`	BERT에 사용된 정규화
`Sequence()`	여러 정규화를 순서대로 수행

사전 토큰화 클래스	설명
`Sequence()`	여러 사전 토큰화를 순서대로 수행
`ByteLevel()`	OpenAI의 GPT-2 방식, 문자열을 그래픽 문자열로 매핑 후 공백 기준 분리
`CharDelimiterSplit()`	문자 기준 분리
`Digits()`	숫자 표현 기준 분리
`Punctuation()`	구두점 기준 분리
`Metaspace()`	공백 기준 + 공백을 replacement('▁')로 치환
`Whitespace()`	단어 경계(공백·구두점) 기준 (정규식 `\w+|[^\w\s]+`)
`WhitespaceSplit()`	공백 문자 기준
`Split(pattern, behavior)`	정규식·동작(removed, isolated, merged_with_previous/next, contiguous) 기준

- 그 외: **BBPE(Byte-level BPE)** (유니코드가 아닌 바이트 단위), **유니그램(Unigram)** (큰 어휘 사전에서 덜 필요한 토큰을 제거하며 학습)

6.0 임베딩 개요

텍스트 벡터화 (Text Vectorization)

- 컴퓨터는 텍스트를 이해할 수 없으므로 **텍스트를 숫자로 변환**하는 과정 필요

- **기초적인 텍스트 벡터화**
  - **원-핫 인코딩(One-Hot Encoding):** 각 단어를 고유 색인에 매핑 → 해당 위치만 1, 나머지 0 (등장 여부만 표시)
    - 예) `I like apples: [1,1,1,0]`, `I like bananas: [1,1,0,1]`
  - **빈도 벡터화(Count Vectorization):** 단어의 **빈도수**를 벡터 값으로 표현
- 장점: 변환이 쉽고 간단
- 단점
  - **희소성(Sparsity)** 이 큼 → 컴퓨팅 비용 ↑, 차원의 저주
  - 벡터가 **의미를 내포하지 않음** (예: 'i scream for ice cream'과 'ice cream for i scream'이 모두 [1,1,1,1,1])
- **해결: 워드 임베딩(Word Embedding)** (Word2Vec, fastText 등)
  - 단어를 **고정된 길이의 실수 벡터**로 표현, 벡터 공간에서 다른 단어와의 **상대적 위치**로 의미 표현
  - 한계: 고정된 임베딩이라 **다의어·문맥 정보** 처리 어려움 → 인공 신경망 기반 **동적 임베딩(Dynamic Embedding)** 사용

6.1 언어 모델

- **언어 모델(Language Model):** 입력된 문장으로 **각 문장을 생성할 수 있는 확률**을 계산하는 모델
  - 자동 번역, 음성 인식, 텍스트 요약 등에 활용
- 문제: '안녕하세요' 뒤에 올 수 있는 문장은 사실상 무한 → **완성된 문장 단위로 확률 계산은 불가능**
- ⇒ **토큰 단위로 예측**하는 **자기회귀 언어 모델** 고안

1) 자기회귀 언어 모델 (Autoregressive Language Model)

- **조건부 확률**을 이용해 이전 단어들의 시퀀스가 주어졌을 때 **다음 단어의 확률**을 계산
- 다음 단어 예측 → 그 출력이 다시 **입력**이 되어 다음 단어 예측 (반복) → 이 특징 때문에 '자기회귀'
- 시점별로 다음 토큰을 예측하므로 **토큰 분류 문제**로 정의 가능
- **조건부 확률의 연쇄법칙(Chain rule)** 적용

$$P(w_1, w_2, \ldots, w_n) = P(w_1),P(w_2|w_1)\cdots P(w_n|w_1, \ldots, w_{n-1})$$

- 문장 전체의 확률 = 첫 단어 확률 × 각 단어가 이전 단어들에 조건부로 발생할 확률의 곱

2) 통계적 언어 모델 (Statistical Language Model)

- 언어의 통계적 구조를 이용해 시퀀스를 생성·분석, 시퀀스 확률 분포를 추정해 다음 단어 확률 예측
- 일반적으로 **마르코프 체인(Markov Chain)** 으로 구현: **빈도 기반 조건부 확률** 모델, 이전 상태와 현재 상태 간 **전이 확률**로 다음 상태 예측
  - 예) '안녕하세요' 1,000번 중 '안녕하세요 만나서' 700번 → P(만나서|안녕하세요) = 700/1000
- **단점:** 순서·빈도에만 의존해 **문맥 파악 부족**, 한 번도 등장하지 않은 단어/문장은 예측 어려움 → **데이터 희소성(Data sparsity)** 문제

- **장점:** 패턴을 찾아 확률 분포를 생성하므로 **새로운 문장 생성** 가능, 대규모 자연어 데이터 처리에 효과적
- 최근에는 언어 모델로 가중치를 **사전 학습**
  - **GPT(Generative Pre-trained Transformer):** 인과적 언어 모델 → 앞부분을 입력받아 **뒤를 예측**
  - **BERT(Bidirectional Encoder Representations from Transformers):** 마스킹된 언어 모델 → `[MASK]` 부분을 예측
  - (자세한 내용은 7장)

3) N-gram

- 가장 기초적인 통계적 언어 모델. **연속된 N개의 단어 시퀀스를 하나의 단위**로 취급해 등장 확률 추정

N	이름
1	유니그램(Unigram)
2	바이그램(Bigram)
3	트라이그램(Trigram)
4 이상	N-gram

- 모든 토큰이 아닌 **N−1개의 이전 토큰만** 고려해 확률 계산: $P(w_n | w_{n-1}, w_{n-2}, \ldots, w_{n-N+1})$
- N 값을 조정해 성능 조절
- 구현: 간단한 파이썬 코드(`zip(*[words[i:] for i in range(n)])`) 또는 `nltk.ngrams` (결과 동일)
- 활용: 작은 규모 데이터의 **연속 문자열 패턴 분석**, **관용적 표현 분석**(예: '입이 무겁다' = 비밀을 잘 지킨다), 단어 순서가 중요한 작업

6.2 TF-IDF

- **TF-IDF(Term Frequency-Inverse Document Frequency):** 문서 내 **특정 단어의 중요도**를 계산하는 통계적 가중치 → **BoW에 가중치를 부여**하는 방법

> 💡 **BoW (Bag-of-Words)**
> : 문서를 **단어의 집합**으로 표현, 중복을 허용해 **등장 빈도**를 기록 (원-핫은 등장 여부만 0/1)
>
> - **한계:** 모든 단어가 **동일한 가중치** → 'I like this movie'와 'I don't like this movie'에서 결정적인 'don't'가 드물게 등장해 무시될 수 있음

개념	정의	수식	해석
단어 빈도 (TF)	문서 내 특정 단어의 **빈도 수**	$TF(t,d) = count(t,d)$	값이 높으면 중요하다고 볼 수 있으나, 전문 용어·관용어일 수도 있고 **문서가 길수록 값이 커짐**
문서 빈도 (DF)	한 단어가 **몇 개의 문서**에 나타나는지	$DF(t,D) = count(t \in d : d \in D)$	높으면 일반적으로 널리 쓰여 **중요도 ↓**, 낮으면 특정 문맥에서만 쓰여 **중요도 ↑**
역문서 빈도 (IDF)	전체 문서 수 ÷ 문서 빈도에 **로그**를 취한 값	$IDF(t,D) = \log\frac{count(D)}{}	단어가 **적게 등장할수록 IDF ↑**

개념	정의	수식	해석
		{1+DF(t,D)}$	
TF-IDF	TF × IDF	$TF\text{-}IDF(t,d,D) = TF(t,d) \times IDF(t,D)$	한 문서에서 자주 등장하지만 전체 문서에서는 드문 단어일수록 **값 ↑**

- 예) 3개 문서에서 'movie'가 4번 등장 → **TF = 4, DF = 3**
- **IDF에서 분모에 1을 더하는 이유:** 단어가 한 번도 등장하지 않아 **분모가 0이 되는 것을 방지**
- **IDF에서 로그를 취하는 이유:** 전체 문서 수 ÷ DF는 너무 큰 값이 나올 수 있음 (10,000개 문서에서 1번 등장 → 5,000) → 로그로 **정교한 가중치** 확보
- 결과적으로 관사·관용어처럼 전체 문서에서 자주 등장하는 단어의 가중치는 **낮아짐**

사이킷런 TfidfVectorizer

- `sklearn.feature_extraction.text.TfidfVectorizer` (`pip install scikit-learn`)

매개변수	의미
`input`	입력 형태 (content: 문자열/바이트, file: 파일 객체, filename: 파일 경로)
`encoding`	바이트·파일 입력 시 텍스트 인코딩
`lowercase`	소문자 변환 여부
`stop_words`	**불용어**(분석에 도움 안 되는 단어), 단어 사전에 추가되지 않음
`ngram_range`	사용할 N-gram 범위 `(최솟값, 최댓값)`, (1,1)은 유니그램, (1,2)는 유니+바이그램
`max_df`	일정 횟수/비율을 **초과**해 등장하는 단어를 불용어 처리 (정수: 횟수, 1 이하 실수: 비율)
`min_df`	일정 횟수/비율 **미만**으로 등장하는 단어를 불용어 처리
`vocabulary`	미리 구축한 단어 사전 (미입력 시 학습 중 자동 구축)
`smooth_idf`	IDF 계산 시 **분모에 1을 더함**

- 사용 흐름: `fit`(학습) → `transform`(변환) 또는 `fit_transform`(동시 수행)
- `toarray()`: 넘파이 배열로 변환, shape = **(문서 수) × (단어 수)** (행: 문서, 열: 단어)
- `vocabulary_`: 단어 사전 (키: 단어, 값: 색인)
- 예) 문서별 TF-IDF 상위 3개 단어: [famous, is, movie] / [actor, like, that] / [actor, don, like] → **문서 내 핵심 단어 추출** 가능
- **한계**
  - 빈도 기반 벡터화는 **문장의 순서·문맥을 고려하지 않음** → 문장 생성 같은 작업에 부적합
  - 벡터는 문서 내 **중요도**일 뿐 **단어의 의미를 담지 않음**
  - 예) '나는 바나나를 먹었다', '그녀가 과일을 섭취한다', '고양이는 야옹 운다' → 앞의 두 문장이 의미상 더 가깝지만 세 문장 간 **유사도가 동일**